Assignment 3

Name- Vedant Wagh
Roll no - C47

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 10   // size of hash table

typedef struct {
    int key;            // position variable (unique integer key)
    char name[30];
    char phone[15];
    int occupied;       // flag to check if slot is occupied
} Client;

Client hashTable[MAX];

// Initialize hash table
void initTable() {
    for (int i = 0; i < MAX; i++) {
        hashTable[i].occupied = 0;
    }
}

// Primary hash function
int hashFunction(int key) {
    return key % MAX;
}

// Secondary hash function for double hashing
int hashFunction2(int key) {
    return 7 - (key % 7);
}

// Insert using linear probing
void insertLinear(int key, char name[], char phone[]) {
    int index = hashFunction(key);
    int i = 0;
    while (hashTable[(index + i) % MAX].occupied && i < MAX) {
        i++;
    }
    if (i == MAX) {
        printf("Hash table full!\n");
        return;
    }
    index = (index + i) % MAX;
    hashTable[index].key = key;
    strcpy(hashTable[index].name, name);
    strcpy(hashTable[index].phone, phone);
```

```
    hashTable[index].occupied = 1;
    printf("Inserted at index %d using Linear Probing\n", index);
}

// Insert using quadratic probing
void insertQuadratic(int key, char name[], char phone[]) {
    int index = hashFunction(key);
    int i = 0;
    while (hashTable[(index + i * i) % MAX].occupied && i < MAX) {
        i++;
    }
    if (i == MAX) {
        printf("Hash table full!\n");
        return;
    }
    index = (index + i * i) % MAX;
    hashTable[index].key = key;
    strcpy(hashTable[index].name, name);
    strcpy(hashTable[index].phone, phone);
    hashTable[index].occupied = 1;
    printf("Inserted at index %d using Quadratic Probing\n", index);
}

// Insert using double hashing
void insertDoubleHash(int key, char name[], char phone[]) {
    int index = hashFunction(key);
    int step = hashFunction2(key);
    int i = 0;
    while (hashTable[(index + i * step) % MAX].occupied && i < MAX) {
        i++;
    }
    if (i == MAX) {
        printf("Hash table full!\n");
        return;
    }
    index = (index + i * step) % MAX;
    hashTable[index].key = key;
    strcpy(hashTable[index].name, name);
    strcpy(hashTable[index].phone, phone);
    hashTable[index].occupied = 1;
    printf("Inserted at index %d using Double Hashing\n", index);
}

// Search for a client by key
void search(int key) {
    for (int i = 0; i < MAX; i++) {
        if (hashTable[i].occupied && hashTable[i].key == key) {
            printf("Found: Name=%s, Phone=%s at index %d\n", hashTable[i].name,
hashTable[i].phone, i);
            return;
        }
    }
    printf("Client not found!\n");
}
```

```
// Display hash table
void display() {
    printf("\nHash Table:\n");
    for (int i = 0; i < MAX; i++) {
        if (hashTable[i].occupied) {
            printf("Index %d: Key=%d, Name=%s, Phone=%s\n", i, hashTable[i].key,
hashTable[i].name, hashTable[i].phone);
        } else {
            printf("Index %d: Empty\n", i);
        }
    }
}

int main() {
    int choice, key;
    char name[30], phone[15];
    initTable();

    while (1) {
        printf("\n--- Telephone Book Menu ---\n");
        printf("1. Insert (Linear Probing)\n");
        printf("2. Insert (Quadratic Probing)\n");
        printf("3. Insert (Double Hashing)\n");
        printf("4. Search Client\n");
        printf("5. Display Table\n");
        printf("6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter key (integer): ");
                scanf("%d", &key);
                printf("Enter name: ");
                scanf("%s", name);
                printf("Enter phone: ");
                scanf("%s", phone);
                insertLinear(key, name, phone);
                break;
            case 2:
                printf("Enter key (integer): ");
                scanf("%d", &key);
                printf("Enter name: ");
                scanf("%s", name);
                printf("Enter phone: ");
                scanf("%s", phone);
                insertQuadratic(key, name, phone);
                break;
            case 3:
                printf("Enter key (integer): ");
                scanf("%d", &key);
                printf("Enter name: ");
                scanf("%s", name);
```

```
            printf("Enter phone: ");
            scanf("%s", phone);
            insertDoubleHash(key, name, phone);
            break;
        case 4:
            printf("Enter key to search: ");
            scanf("%d", &key);
            search(key);
            break;
        case 5:
            display();
            break;
        case 6:
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
  }
  return 0;
}
```